

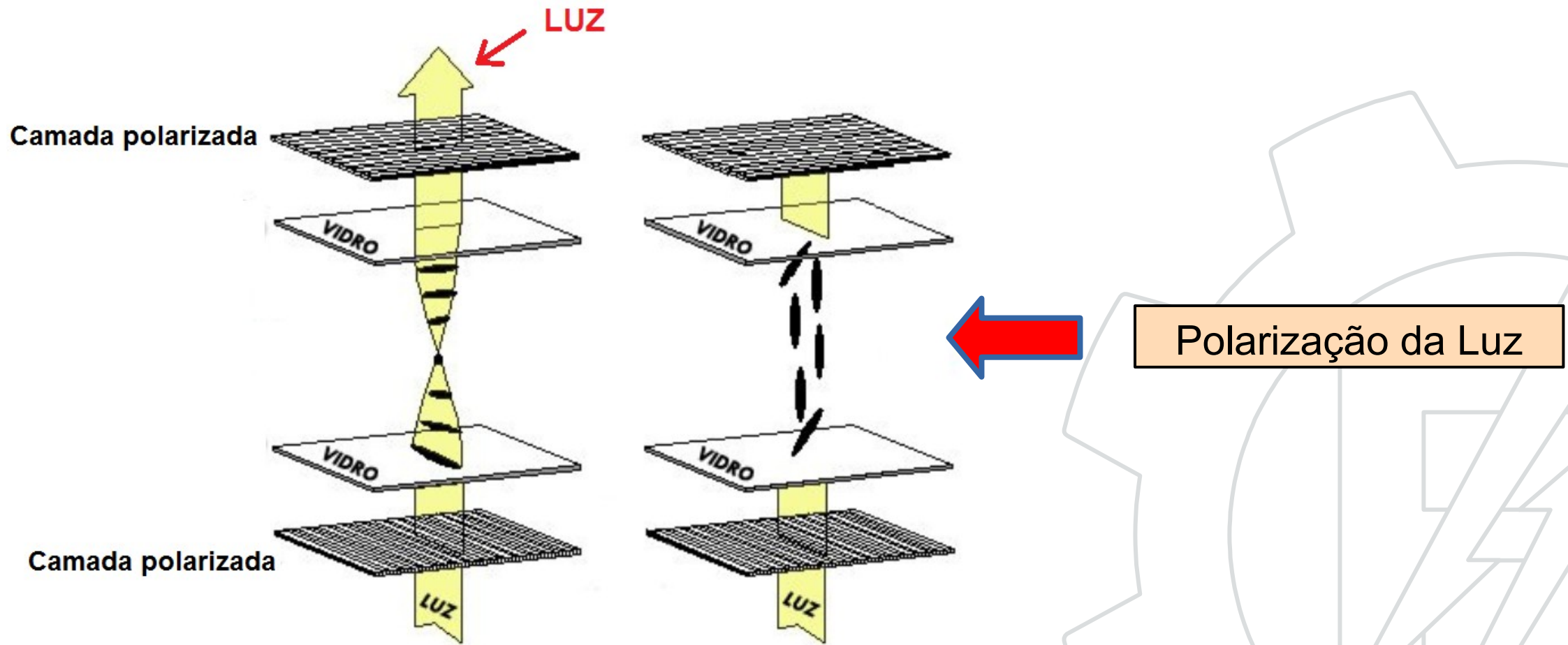
Display de LCD



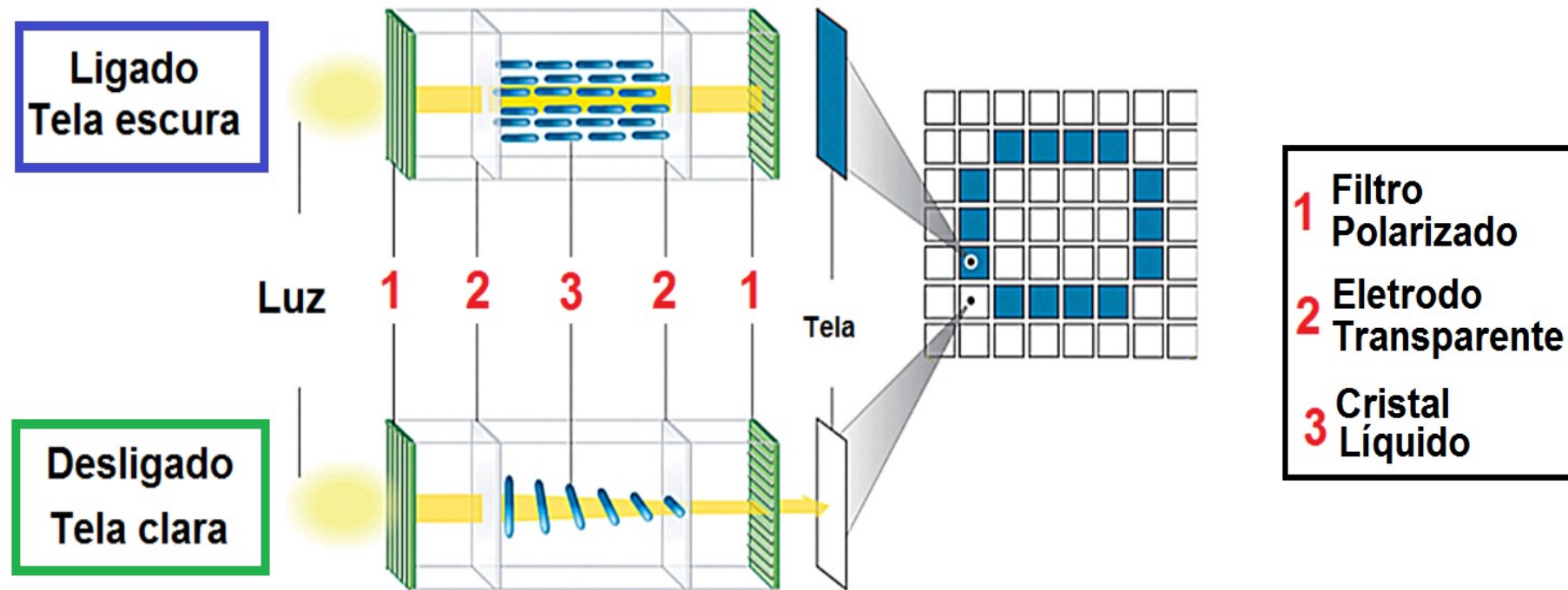
Display de LCD



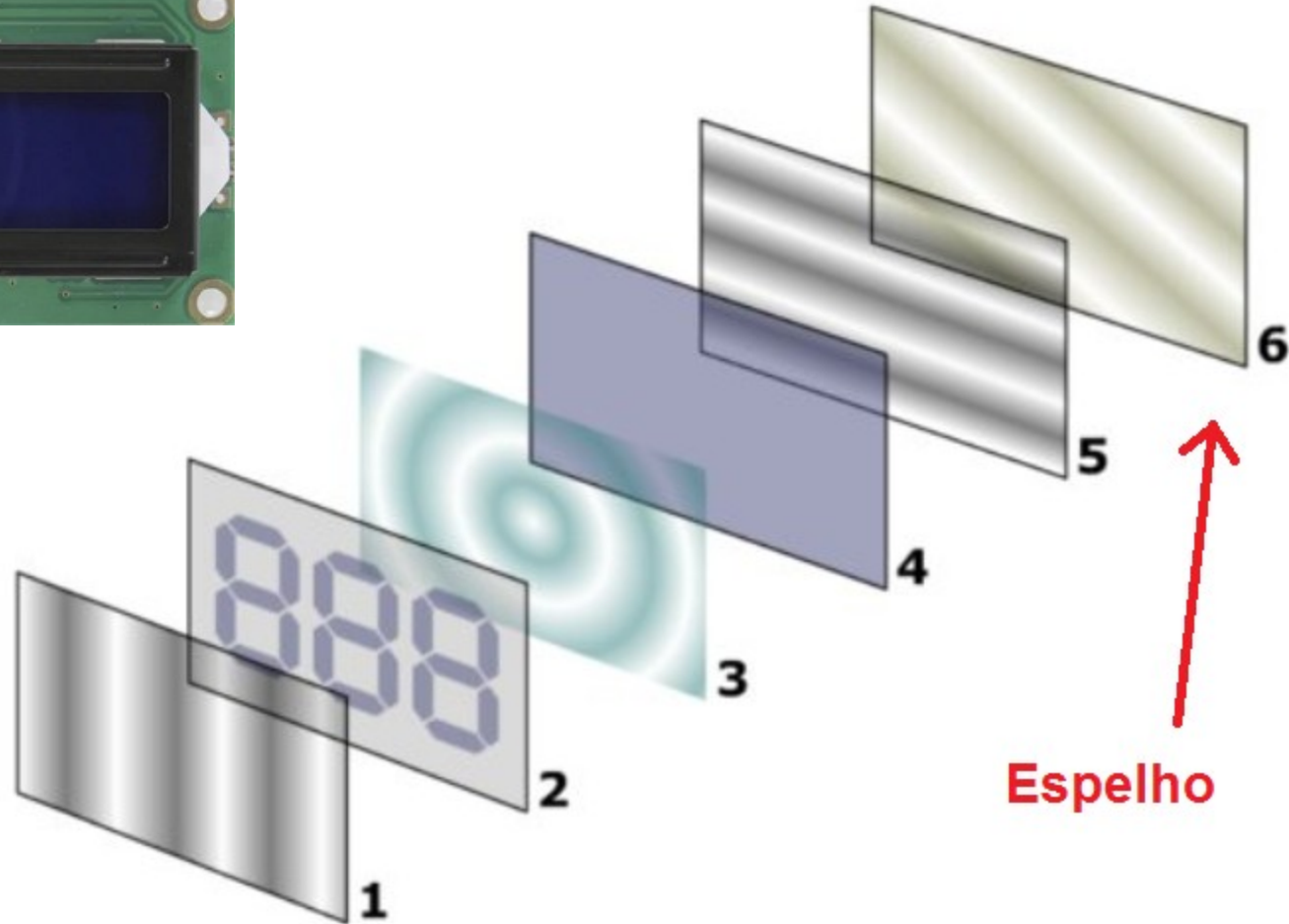
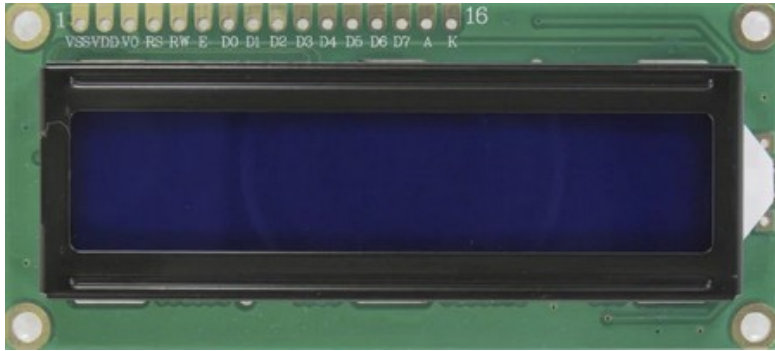
Princípio de funcionamento



Princípio de funcionamento

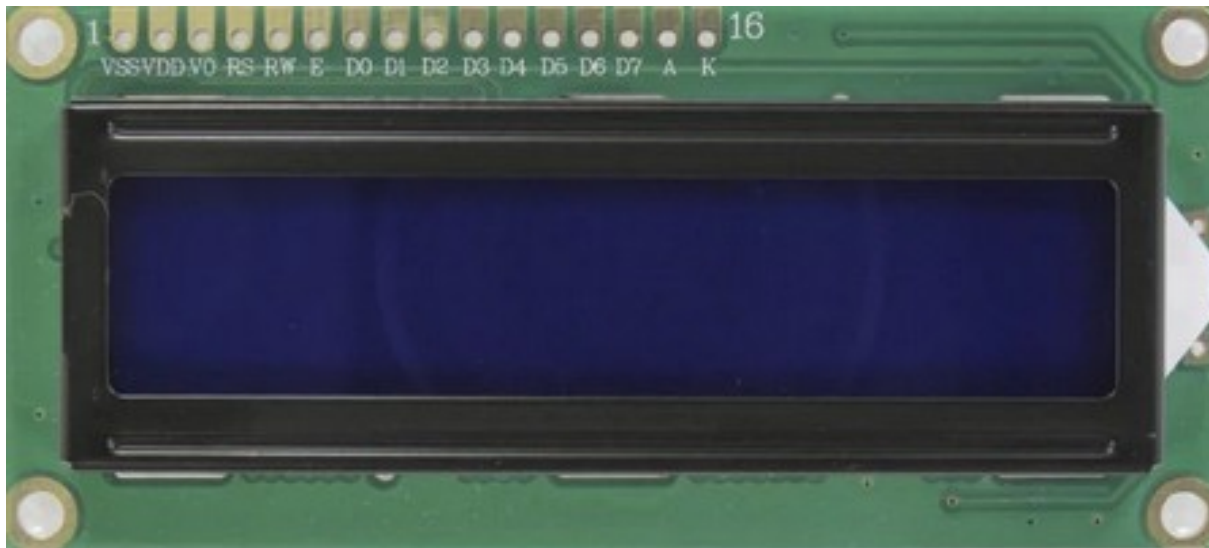


Princípio de funcionamento

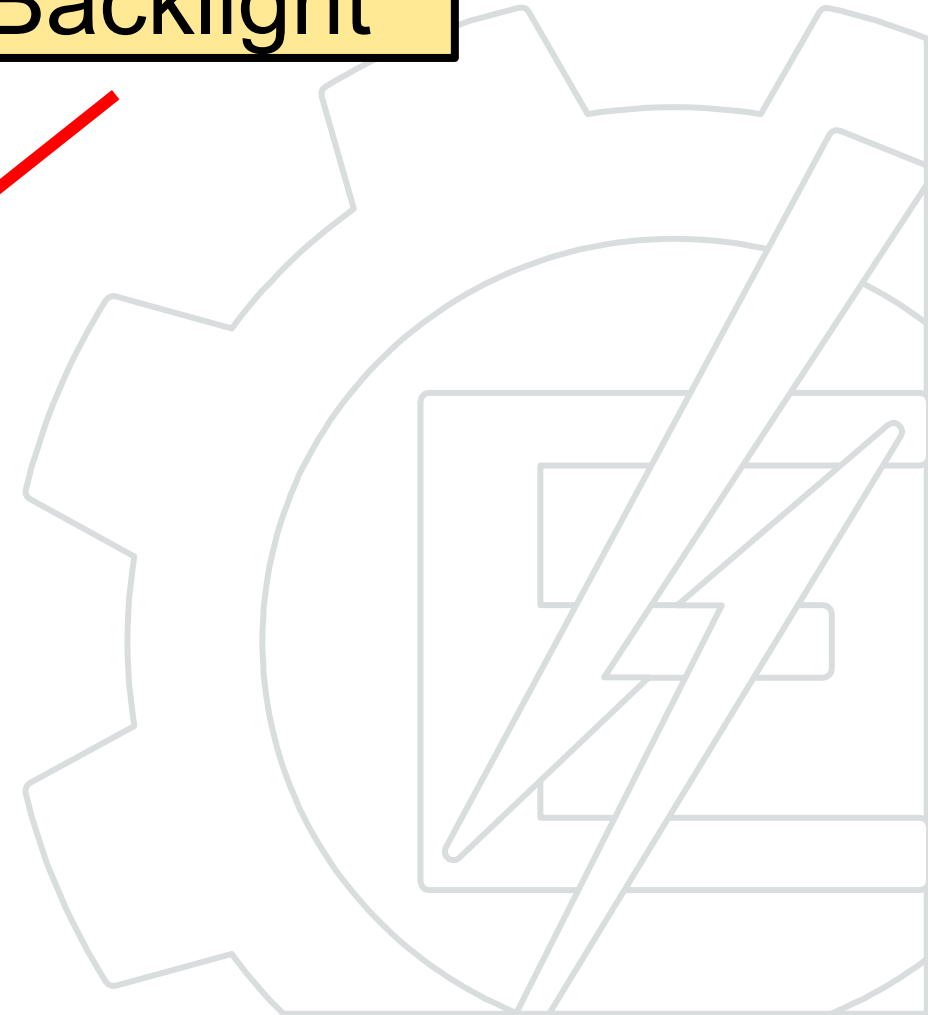
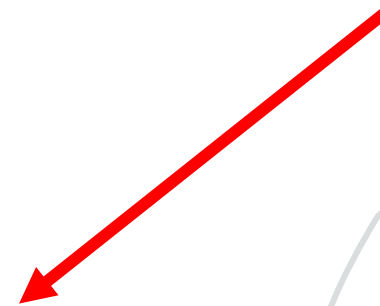


Espelho

Princípio de funcionamento

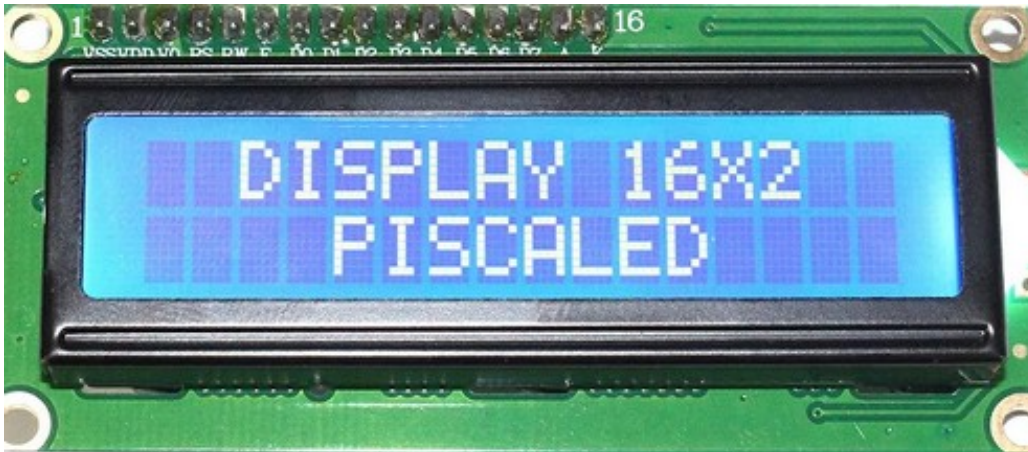


Backlight



Formação das imagens

O formato mais comum é o matricial 5x8 que permite a formação de letras números e símbolos.



Matriz 5x8



→ 0b000 00000
→ 0b000 01010
→ 0b000 01010
→ 0b000 01010
→ 0b000 00000
→ 0b000 10001
→ 0b000 01110
→ 0b000 00000

Controle

Na maioria das vezes é fornecido com um microcontrolador e funciona como uma placa de vídeo.

Microcontrolador integrado



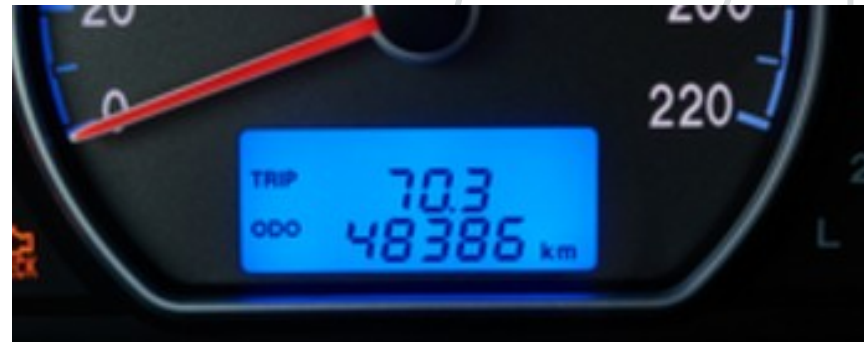
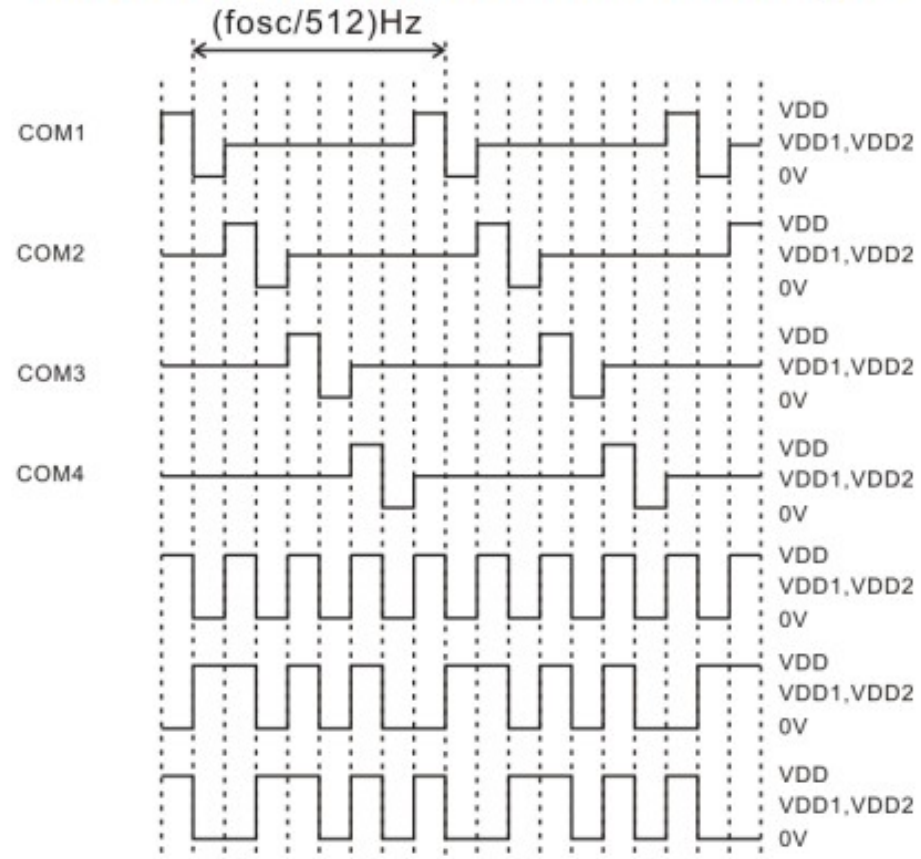
Aplicações especiais

LCD dedicados



Controle

1/4 DUTY, 1/2 BIAS DRIVE TECHNIQUE



Tipos mais comuns



Texto 16x2

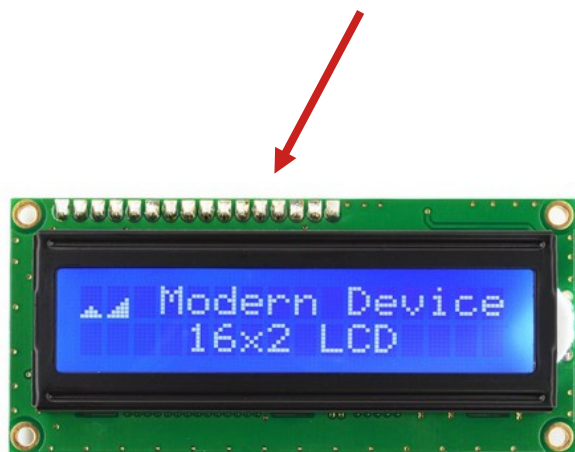
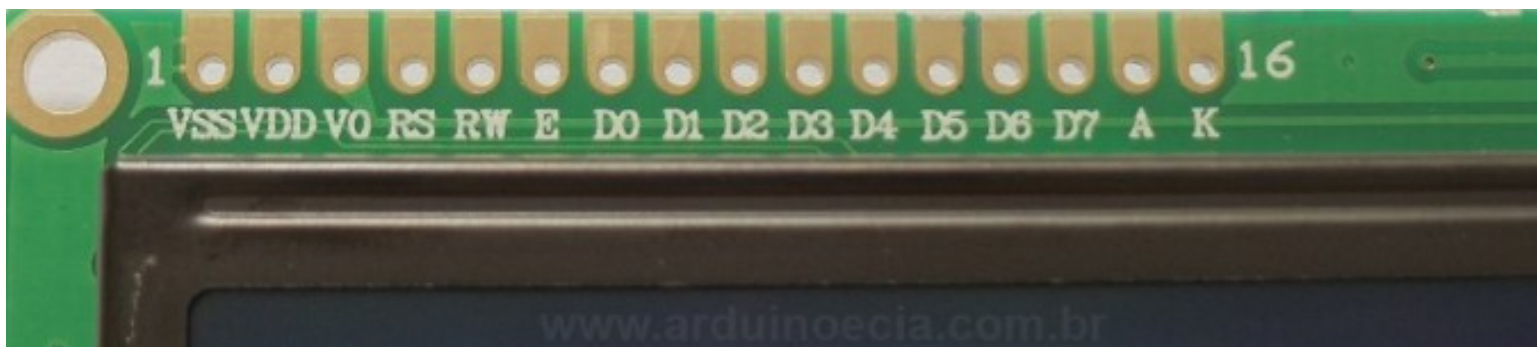


Gráfico 128x64



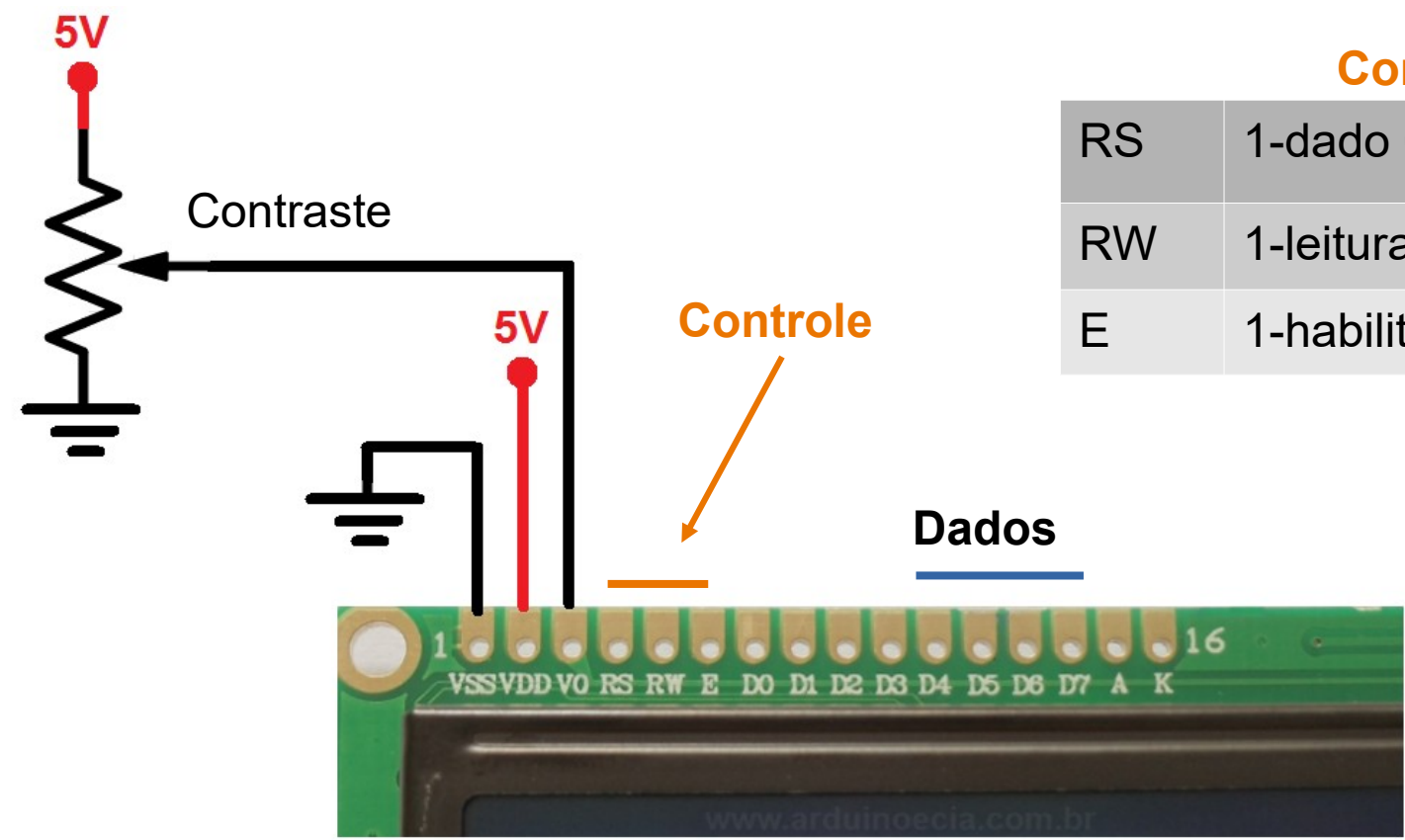
OLED 128x64

Controle



Controlador HD44780

Controle



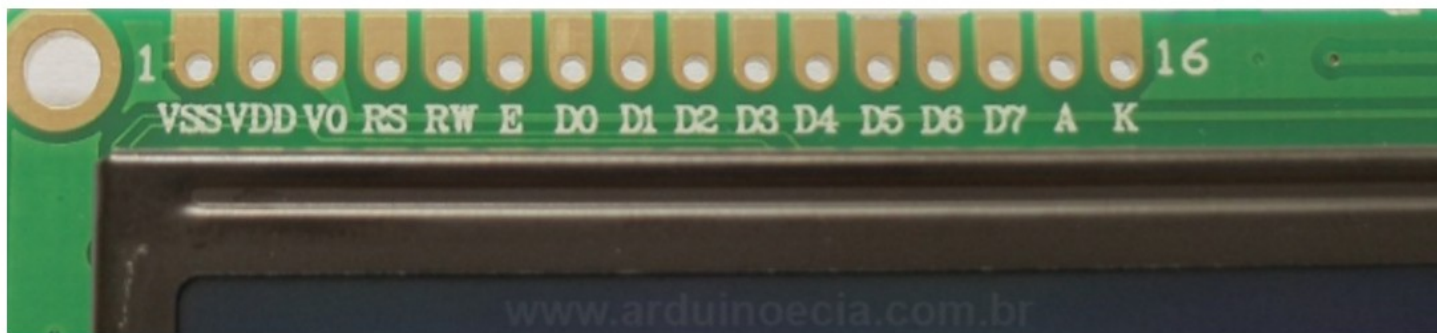
Controle

RS	1-dado 0-instrução
RW	1-leitura 0-escrita
E	1-habilita 0-desabilita

Controle

Enable

Leitura ou escrita é realizada
quando habilitado (1)

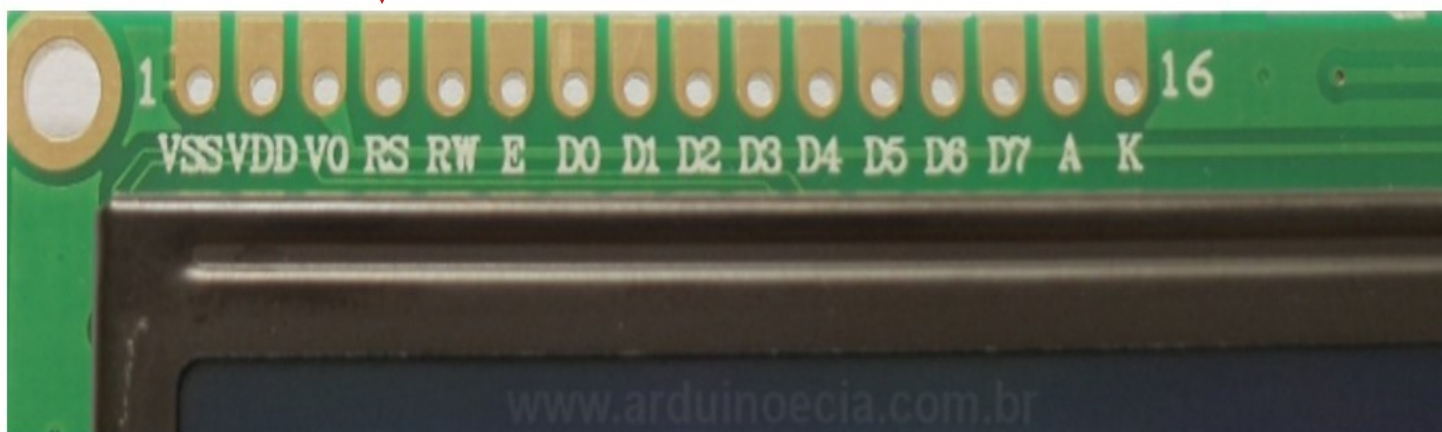


Controle

RS



1 - Dado
0 - Comando

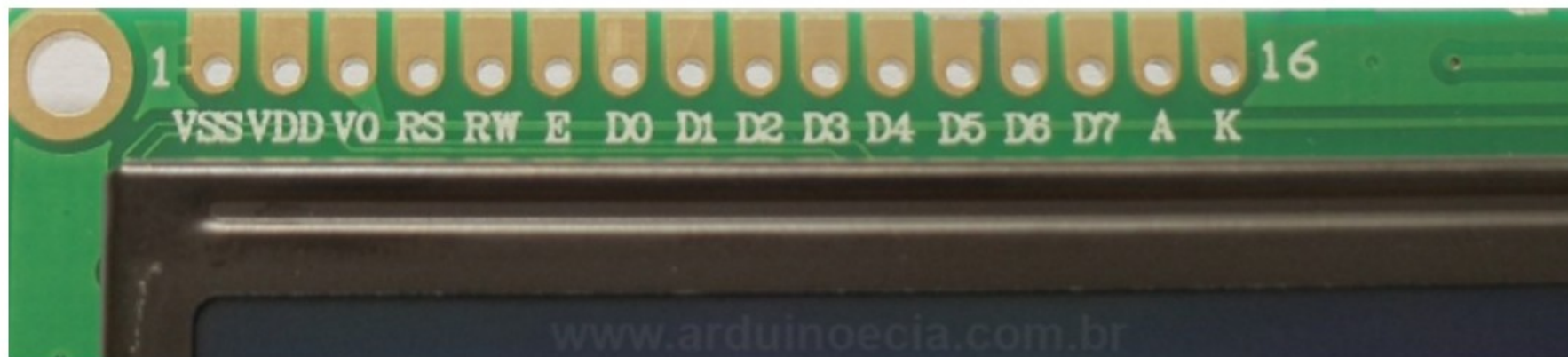


Controle

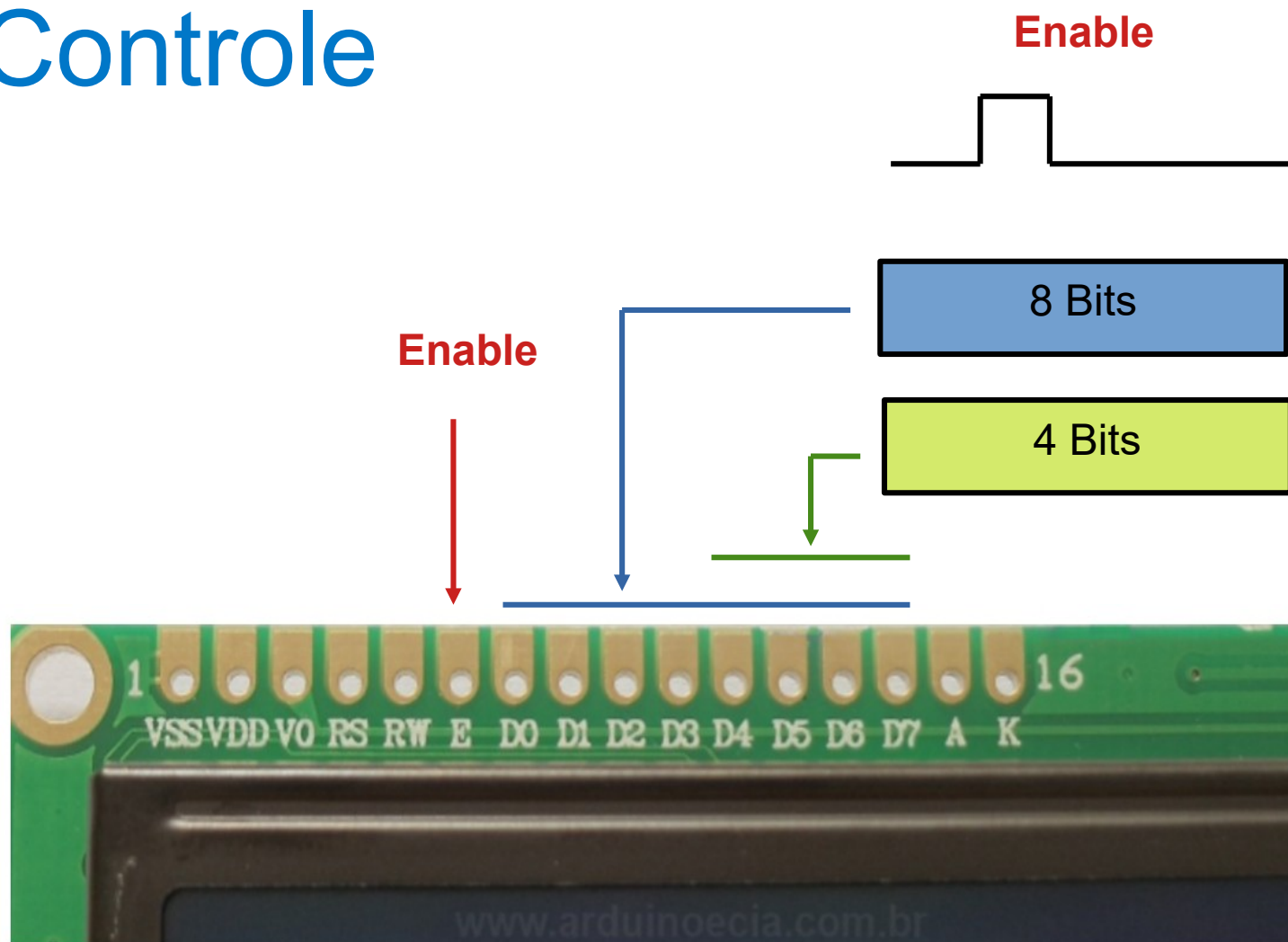
RW



1 - Leitura
0 - Escrita



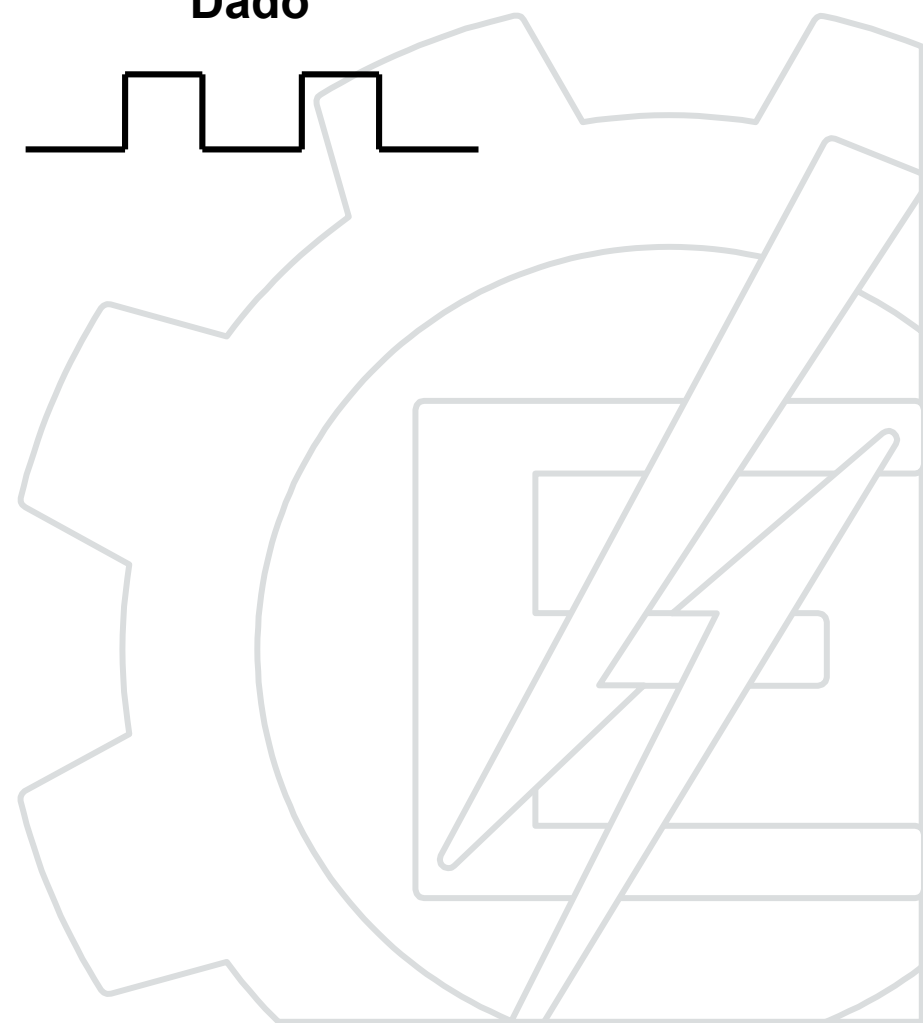
Controle



Enable



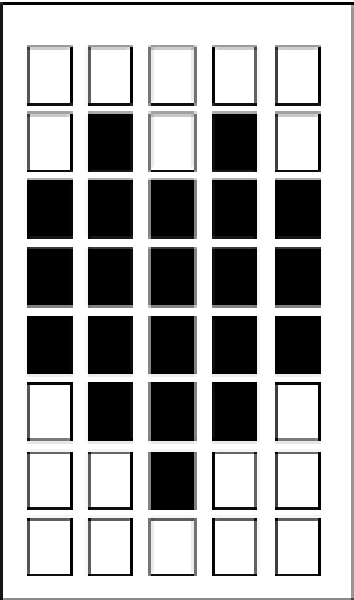
Dado



Controle

Lower 4 Bits	Upper 4 Bits	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000		CG RAM (1)	▶		0	Q	P	`	P	B	α		°	À	Ð	à	ä
	(2)	◀	!	1	A	Q	a	q	A	J	i	±	Á	Ñ	á	ñ	
xxxx0010	(3)	“	”	2	B	R	b	r	W	Γ	φ	²	Â	Ò	â	ò	
	(4)	”	#	3	C	S	c	s	3	π	£	³	Ã	Ó	ã	ó	
xxxx0100	(5)	⌂	\$	4	D	T	d	t	M	Σ	¥	₣	Ä	Ö	ä	ö	
	(6)	₹	%	5	E	U	e	u	Ï	σ	¥	₤	Å	Ø	å	ø	
xxxx0110	(7)	●	&	6	F	V	f	v	J	¼	¡	¢	Æ	Ö	æ	ö	
	(8)	¶	'	7	G	W	g	w	Π	τ	§	•	Ç	×	ç	÷	
xxxx1000	(1)	↑	(	8	H	X	h	x	Y	‡	¶	ω	È	Θ	è	θ	
	(2)	↓	)	9	I	Y	i	y	U	Θ	¹	°	É	Ù	é	ù	
xxxx1010	(3)	→	*	:	J	Z	j	z	4	Ω	∂	Ω	Ê	Ú	ê	ú	
	(4)	←	+	;	K	[	k	[	W	δ	«	»	Ë	Û	ë	û	
xxxx1100	(5)	≤	,	<	L	\	l		W	∞	№	¼	Ì	Ü	ì	ü	
	(6)	≥	-	=	M	]	m	>	b	¶	¥	½	Í	Ý	í	ý	
xxxx1110	(7)	▲	.	>	N	^	n	~	b	ε	Ω	¾	Î	Þ	î	þ	
	(8)	▼	/	?	O	_	o	ó	Ω	Π	‘	¿	Ï	ß	ï	ÿ	

Matriz 5x8



ASCII

Exemplo: Letra 'A' = 0x41.

Controle



Imagem: www.graphicsfactory.com

Cuidado

Os comandos não são imediatos.

Comando	Tempo
Reset	1,5ms
Limpa	37us
Config	37us

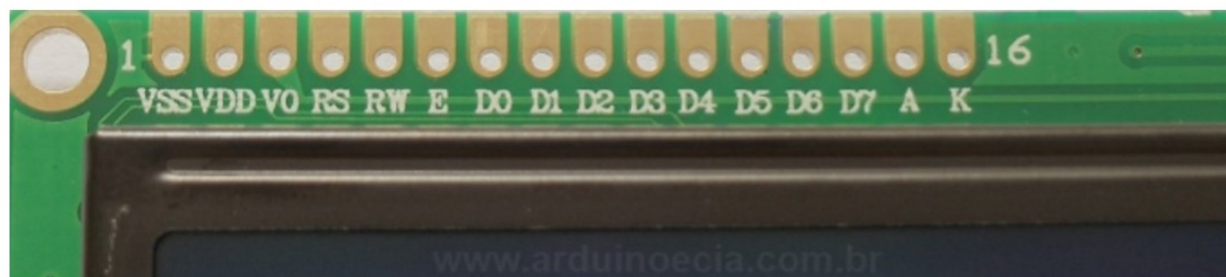


Controle

Resumo

- 1 – Ajustar RS (0 – comando 1 – dado).
- 2 – Habitar escrita (RW = 0).
- 3 – Colocar a informação em dados.
- 4 – Ligar o Enable (E = 1).
- 5 – Dar um pequeno delay.
- 6 – Desligar o Enable (E = 0).
- 7 – Se modo 4 bits repetir o procedimento.
- 8 – Dar um delay maior para o display processar.

RS	1-dado 0-instrução
RW	1-leitura 0-escrita
E	1-habilita 0-desabilita



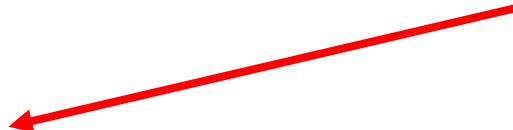
Programando

```
#define _XTAL_FREQ 4000000

// CONFIG
#pragma config FOSC = INTOSCIO // Seleciona oscilador interno
#pragma config WDTE = OFF       // Watchdog Timer desligado
#pragma config PWRTE = OFF      // Power-up Timer desligado
#pragma config MCLRE = OFF      // Pino RA5/MCLR/VPP entrada digital
#pragma config BOREN = OFF      // Brown-out Detect desabilitado
#pragma config LVP = OFF        // Low-Voltage Programming desabilitado
#pragma config CPD = OFF        // Proteção Dados EE Memory desabilitado
#pragma config CP = OFF         // Proteção do código desabilitado

// Definindo os pinos e suas funções
#define LCD_RS_TRIS    TRISBbits.TRISB3
#define LCD_RS         PORTBbits.RB3
#define LCD_E_TRIS     TRISBbits.TRISB0
#define LCD_E          PORTBbits.RB0
#define LCD_D4_TRIS    TRISAbits.TRISA1
#define LCD_D4         PORTAbits.RA1
#define LCD_D5_TRIS    TRISAbits.TRISA0
#define LCD_D5         PORTAbits.RA0
#define LCD_D6_TRIS    TRISAbits.TRISA7
#define LCD_D6         PORTAbits.RA7
#define LCD_D7_TRIS    TRISAbits.TRISA6
#define LCD_D7         PORTAbits.RA6
```

Definindo os nomes dos
pinos utilizados



Programando

```
#include <xc.h>
#include <pic.h>
#include <pic16f628a.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Funções do display
void mostra_string(void);
void envia_nibble_display(unsigned char dado);
void envia_byte_display(unsigned char tipo, int dado);
void inicializa_display(void);
void gotoxy_display(unsigned char linha, unsigned char coluna);
void limpa_display(void);

char str[30]; // Usada para mostrar a mensagem

#define COMANDO 0
#define DADO 1
```

Declaração das funções

Criando macros

Programando

```
// *****  
// Envia um nibble para o display  
// *****  
void envia_nibble_display(unsigned char dado)  
{  
    if(dado&0x01)    LCD_D4 = 1; else    LCD_D4 = 0;  
    if(dado&0x02)    LCD_D5 = 1; else    LCD_D5 = 0;  
    if(dado&0x04)    LCD_D6 = 1; else    LCD_D6 = 0;  
    if(dado&0x08)    LCD_D7 = 1; else    LCD_D7 = 0;  
    LCD_E = 1;  
    __delay_us(1);  
    LCD_E = 0;  
}
```

Envia o nibble

```
// *****  
// Envia um byte para o display  
// tipo = 0 -> comando  
// tipo = 1 -> dado  
// *****  
void envia_byte_display(unsigned char tipo, int dado)  
{  
    LCD_RS = tipo; // Ajusta dado ou instrucao  
    __delay_us(100);  
    envia_nibble_display(dado>>4);    // Envia a parta alta do byte  
    envia_nibble_display(dado & 0x0f); // Envia a parta baixa do byte  
    __delay_us(40);  
}
```

TIPO: Dado ou Comando

Envia byte

Programando

```
- // *****  
// Inicializa o display  
// *****  
void inicializa_display(void)  
{  
    __delay_ms(15);  
    envia_nibble_display(0x03);  
    __delay_ms(5);  
    envia_nibble_display(0x03);  
    __delay_ms(5);  
    envia_nibble_display(0x03);  
    __delay_ms(5);  
    envia_nibble_display(0x02);  
    __delay_ms(1);  
    envia_byte_display(0,0x28);  
    envia_byte_display(0,0x0c);  
    limpa_display();  
    envia_byte_display(0,0x06);  
}
```

Nibble

byte

Inicialização do display

Comandos

- 4 vias ou 8 vias.
- 16x2.
- Limpa display.
- Cursor on/off.
- Blinking on/off.
- Font 5x8.

Programando

Posiciona cursor

```
- // *****  
// Define posição do caracter  
// *****  
void gotoxy_display(unsigned char linha, unsigned char coluna)  
- {  
    int posicao=0;  
    if(linha == 1)  
    {  
        posicao=0x80;    // endereço da linha 1  
    }  
    if(linha == 2)  
    {  
        posicao=0xc0;    // endereço da linha 2  
    }  
    posicao=posicao+coluna-1; // ajusta coluna  
    envia_byte_display(COMANDO,posicao);  
}
```

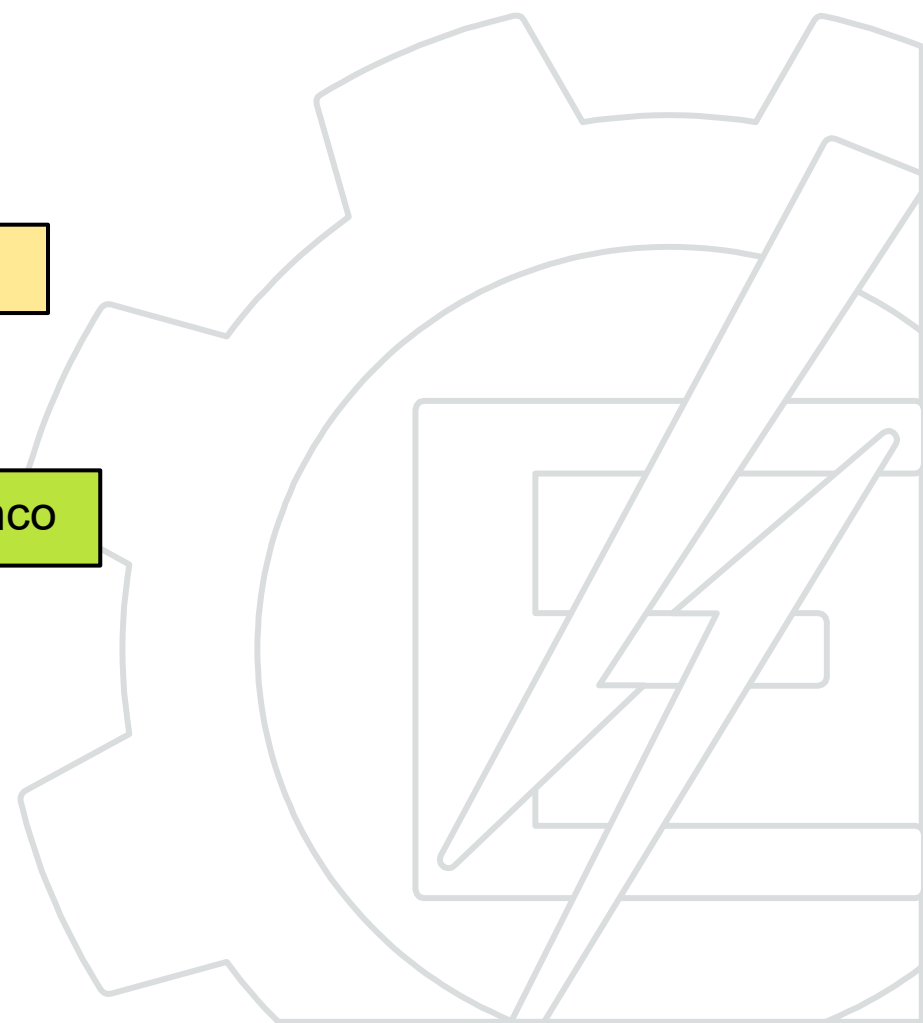
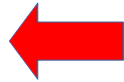
Comando

Programando

```
- // *****  
// Limpa display  
// *****  
void limpa_display(void)  
- {  
  unsigned char x;  
  gotoxy_display(0,0);  
  for(x=0;x<16;x++)  
    envia_byte_display(DADO, ' ');  
  gotoxy_display(1,0);  
  for(x=0;x<16;x++)  
    envia_byte_display(DADO, ' ');  
}
```

Limpa display

Escreve em branco



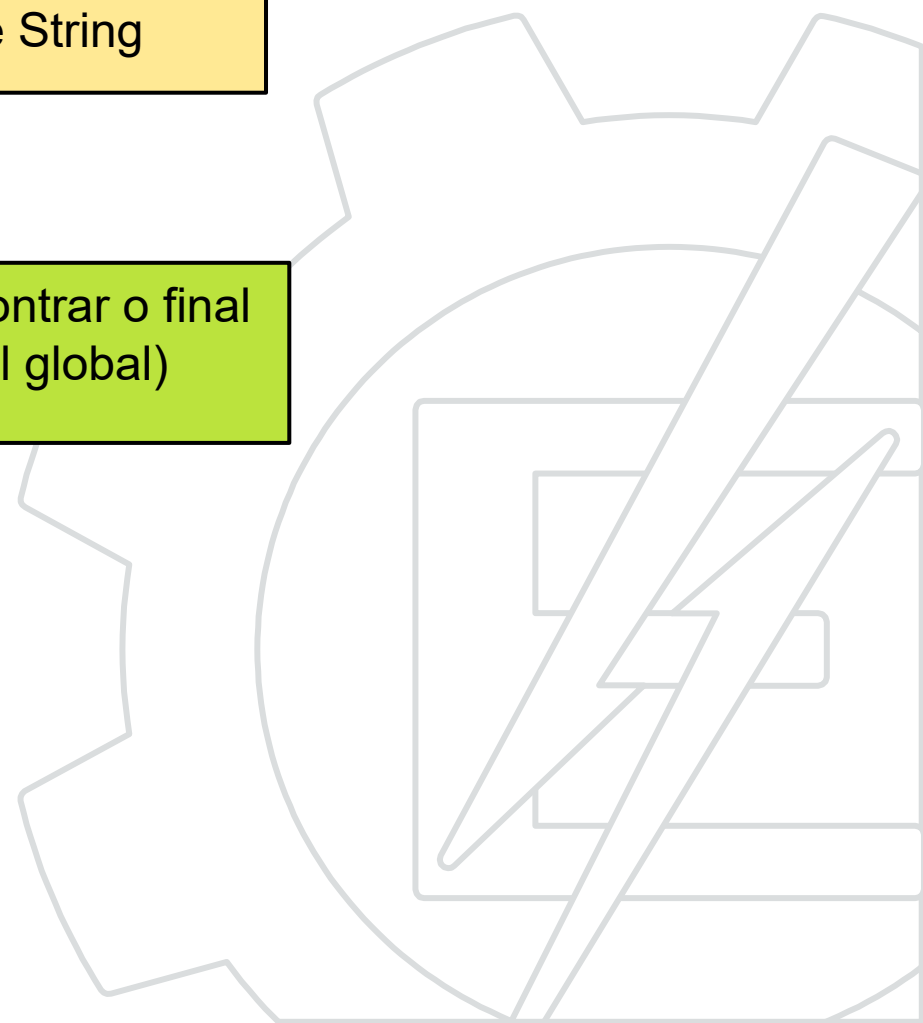
Programando

```
- // *****  
- // Escreve uma string no display  
- // *****  
void mostra_string(void)  
{  
    unsigned char x=0;  
    while( (str[x]!='\0') && (x<16))  
    {  
        envia_byte_display(DADO, str[x]);  
        x++;  
    }  
}
```

Escreve String

Faz um loop até encontrar o final da string (variável global)

Envia letra



Programando

```
void main(void) {  
  
    CMCON = 0x07;  
  
    LCD_RS_TRIS = 0; // Definindo todos os pinos como saída  
    LCD_E_TRIS = 0;  
    LCD_D4_TRIS = 0;  
    LCD_D5_TRIS = 0;  
    LCD_D6_TRIS = 0;  
    LCD_D7_TRIS = 0;  
  
    LCD_RS=0; LCD_E=0; LCD_D4=0; LCD_D5=0; LCD_D6=0; LCD_D7=0;  
  
    while(1){  
        inicializa_display();  
        limpa_display();  
        sprintf(str, "    UNIFEI 2022  ");  
        gotoxy_display(1,1);  
        mostra_string();  
        __delay_ms(10000);  
    }  
}
```

A função main()

Todos os pinos saída
e com nível zero

Enviando a informação

Comandos

Instrução	RS	RW	Bits								Tempo
			7	6	5	4	3	2	1	0	
Limpa	0	0	0	0	0	0	0	0	0	1	37 us
Reset	0	0	0	0	0	0	0	0	1	-	1.5ms
Config.	0	0	0	0	0	0	0	1	ID	S	37 us
Config.	0	0	0	0	0	0	1	D	C	B	37 us
Movim.	0	0	0	0	0	1	SC	RL	-	-	37 us
Config.	0	0	0	0	1	DL	N	F	-	-	37 us
Movim (l,c)	0	0	1	X	0	0	Coluna				37 us
Ocup.	0	1	BF	-	-	-	-	-	-	-	10 us

Opções dos comandos

- ID: 1 -- Incrementa, 0 -- Decrementa
- S: 1 -- O display acompanha o deslocamento
- SC: 1 -- Desloca o display, 0 -- Desloca o cursor
- RL: 1 -- Move para direita, 0 -- Move para esquerda
- DL: 1 -- 8 bits, 0 -- 4 bits
- N: 1 -- 2 linhas, 0 -- 1 linha
- F: 1 -- 5x10 pontos, 0 -- 5x8 pontos
- BF: 1 -- Ocupado, 0 -- Disponível
- X: 1 -- 2ª linha, 0 -- 1ª linha
- Coluna: nibble indicativo da coluna



Exemplo de biblioteca LCD



Biblioteca LCD

Arquivo lcd.h

```
#ifndef LCD_H
#define LCD_H
    void lcdCommand(char value);
    void lcdChar(char value);
    void lcdInit(void);
    void lcdString(char msg[]);
    void lcdNumber(int value);
    void lcdPosition(int line, int col);
#endif
```




```
#include "lcd.h"
```

```
void delayMicro(int a) {  
    volatile int i;  
    for (i = 0; i < (a * 2); i++);  
}
```

```
void delayMili(int a) {  
    volatile int i;  
    for (i = 0; i < a; i++) {  
        delayMicro(1000);  
    }  
}
```

```
//Gera um clock no enable
```

```
void pulseEnablePin() {  
    digitalWrite(LCD_EN_PIN, HIGH);  
    delayMicro(5);  
    digitalWrite(LCD_EN_PIN, LOW);  
    delayMicro(5);  
}
```

Funções auxiliares



```
//Envia 4 bits e gera um clock no enable
void pushNibble(char value, int rs) {
    sOWrite(value);
    digitalWrite(LCD_RS_PIN, rs);
    pulseEnablePin();
}

//Envia 8 bits em dois pacotes de 4
void pushByte(char value, int rs) {
    sOWrite(value >> 4);
    digitalWrite(LCD_RS_PIN, rs);
    pulseEnablePin();

    sOWrite(value & 0x0F);
    digitalWrite(LCD_RS_PIN, rs);
    pulseEnablePin();
}
```



```
void lcdCommand(char value) {  
    pushByte(value, LOW);  
    delayMili(2);  
}
```

Envia comando

```
void lcdChar(char value) {  
    pushByte(value, HIGH);  
    delayMicro(80);  
}
```

Envia caracter

```
void lcdPosition(int line, int col) {
    if (line == 0) { lcdCommand(0x80 + (col % 16)); }
    if (line == 1) { lcdCommand(0xC0 + (col % 16)); }
}
//Imprime um texto (vetor de char)
void lcdString(char msg[]) {
    int i = 0;
    while (msg[i] != 0) {
        lcdChar(msg[i]);
        i++;
    }
}
void lcdNumber(int value) {
    int i = 10000; //Máximo 99.999
    while (i > 0) {
        lcdChar((value / i) % 10 + 48);
        i /= 10;
    }
}
```

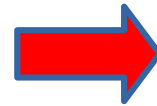
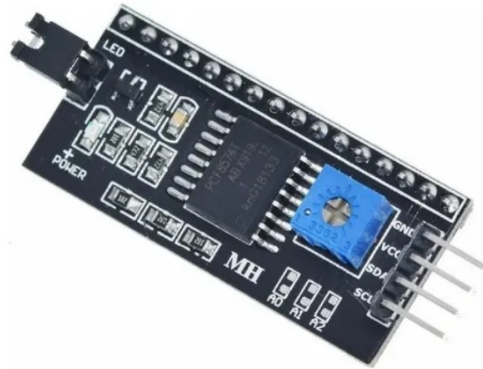
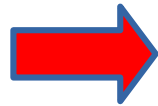
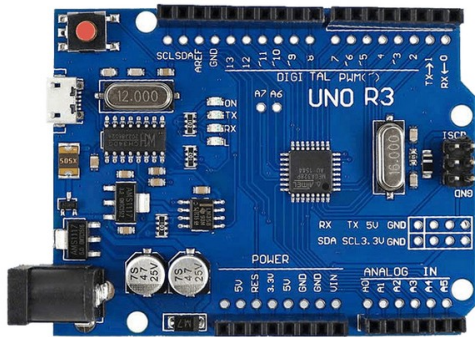


```
// Rotina de inicialização
void lcdInit() {
    pinMode(LCD_EN_PIN, OUTPUT);
    pinMode(LCD_RS_PIN, OUTPUT);
    soInit();
    delayMili(15);
    // Comunicação começa em estado incerto
    pushNibble(0x03, LOW);
    delayMili(5);
    pushNibble(0x03, LOW);
    delayMicro(160);
    pushNibble(0x03, LOW);
    delayMicro(160);
    // Mudando comunicação para 4 bits
    pushNibble(0x02, LOW);
    delayMili(10);
    // Configura o display
    lcdCommand(0x28);           //8bits, 2 linhas, fonte: 5x8
    lcdCommand(0x08 + 0x04);    //display on
    lcdCommand(0x01);           //limpar display, posição 0
}
```



Módulo I2C para LCD

Conversor I2C

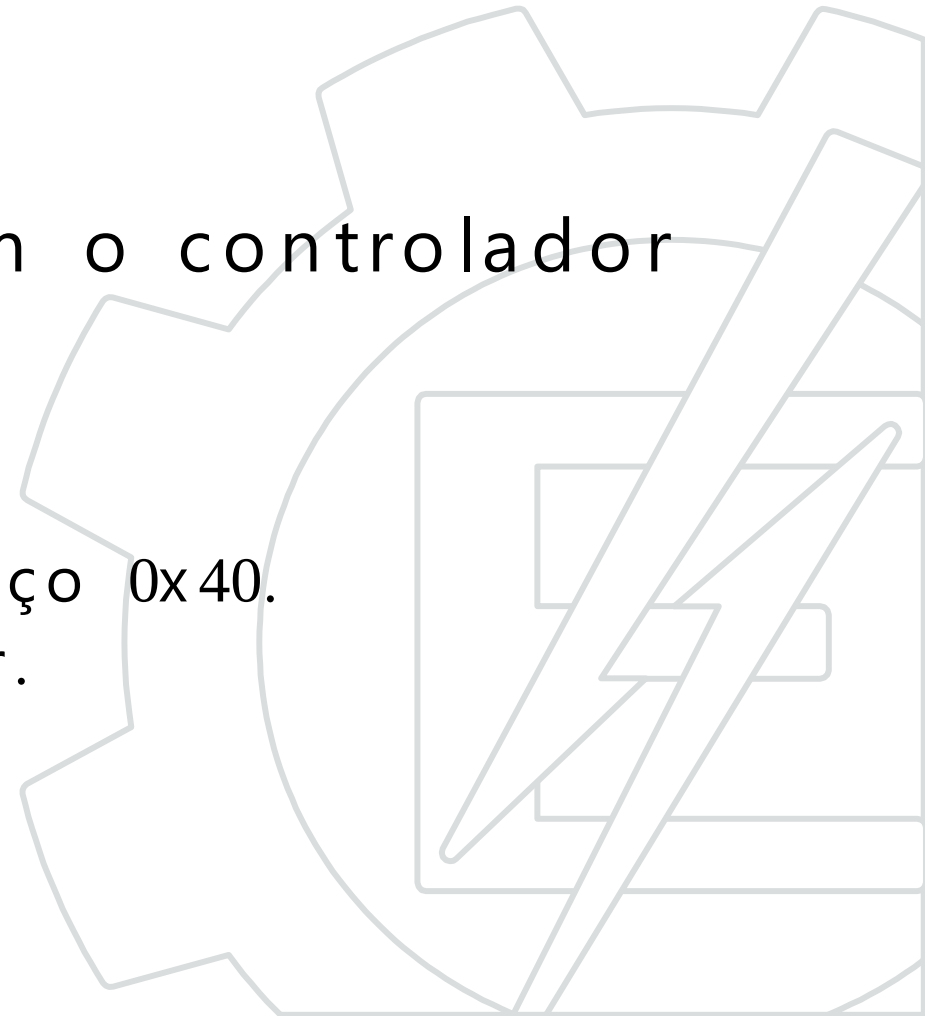


Desenhando
imagens no LCD



Desenhando imagens no LCD

- A maioria dos LCD's permite a criação de caracteres customizados.
- Para os displays compatíveis com o controlador HD 44780 temos:
 - 8 caracteres disponíveis.
 - Formato binário e matricial de 8*5.
 - Armazenamento a partir do endereço 0x40.
 - Valores salvos por linha de caracter.

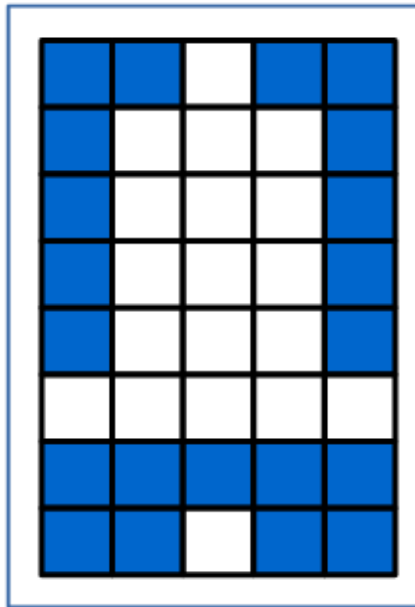


Desenhando imagens no LCD

Controlador HD44780

0x40	Caracter 0
0x47	
0x48	Caracter 1
0x4F	
0x50	Caracter 2
0x57	
	...
0x70	Caracter 6
0x77	
0x78	Caracter 7
0x7F	

0x50	1a linha	0x04
0x51	2a linha	0x0E
0x52	3a linha	0x0E
0x53	4a linha	0x0E
0x54	5a linha	0x0E
0x55	6a linha	0x1F
0x56	7a linha	0x00
0x57	8a linha	0x04



5 Bits por linha

Enviando o símbolo para o lcd

```
//Cada linha é representada por um char (8 bits)
char sino[8] = {0x04, 0x0E, 0x0E, 0x0E,
                0x0E, 0x1F, 0x00, 0x04};

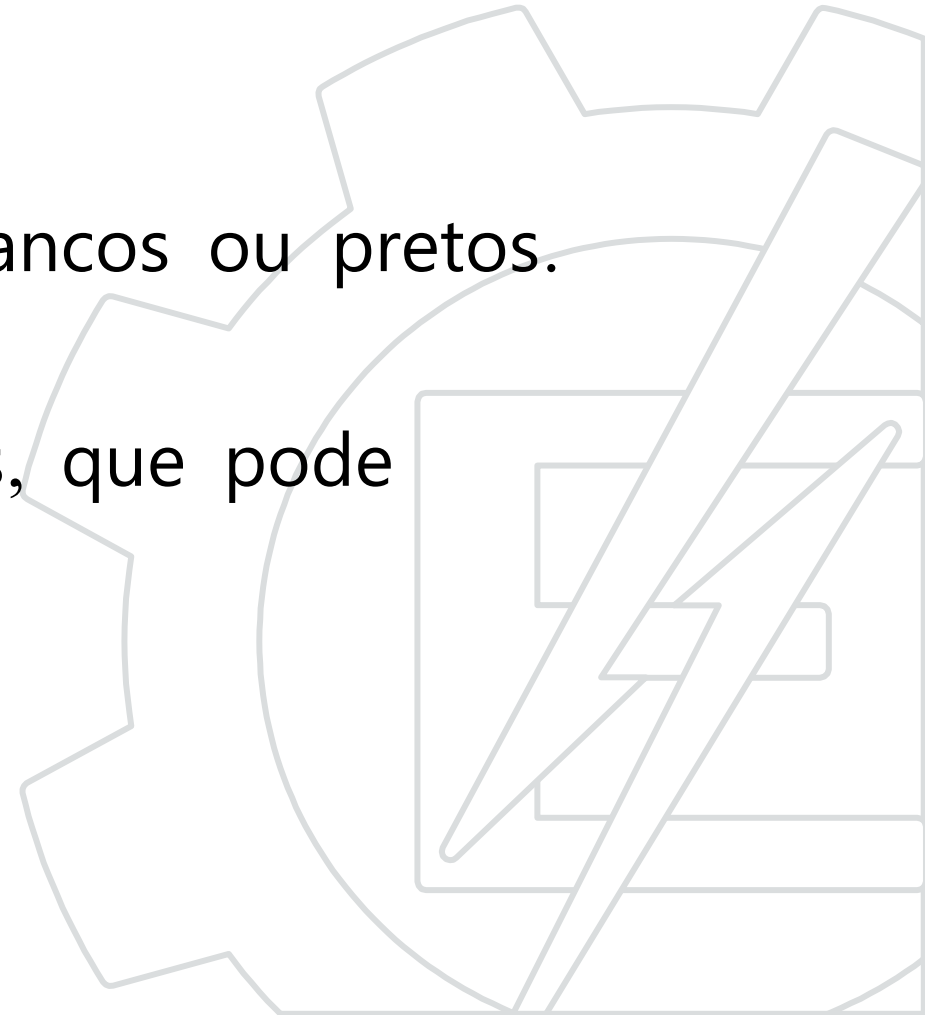
//Configura para a primeira posição de memória

lcdCommand(0x40);
//Envia cada uma das linhas em ordem
for(i=0; i<8; i++){
    lcdChar(sino[i]);
}
```



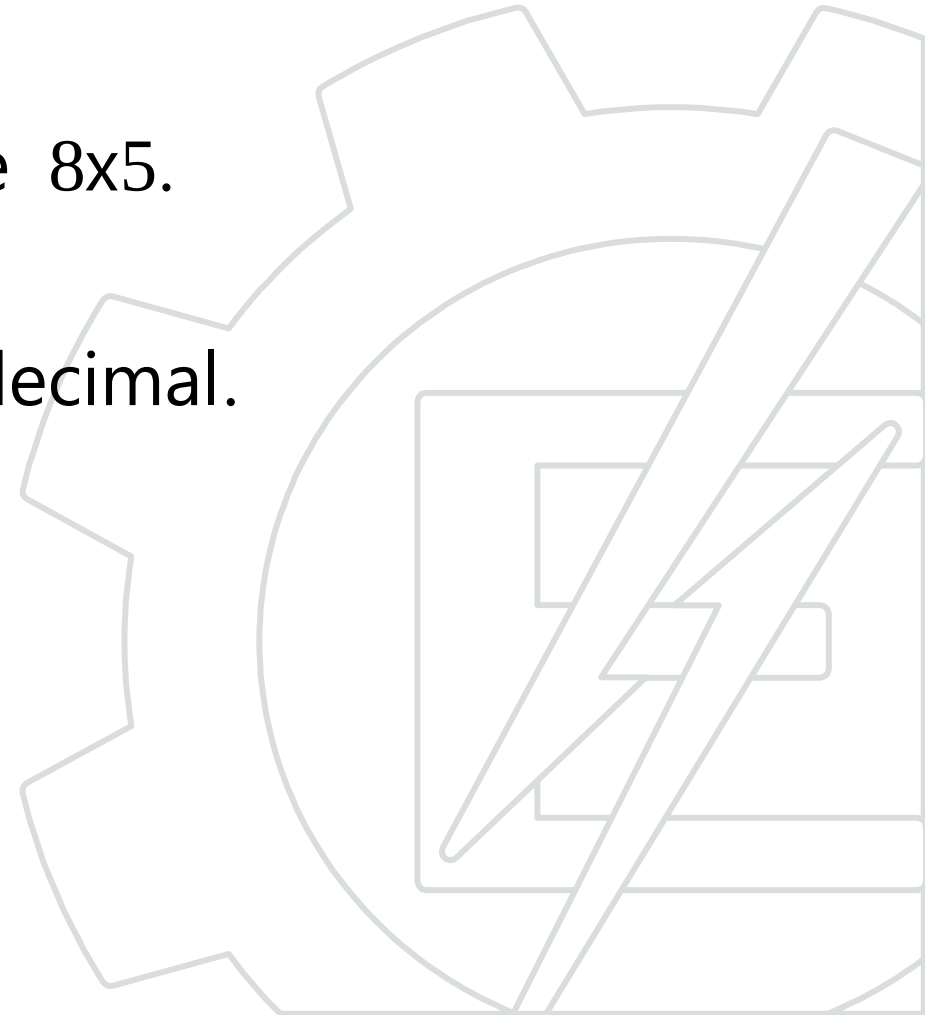
Desenhando imagens no LCD

- É possível criar um desenho/imagem de até 20×16 pixels (4×2 caracteres).
- A imagem será binária, apenas pixels brancos ou pretos.
- Existe uma separação entre os caracteres, que pode prejudicar de certa maneira a imagem.

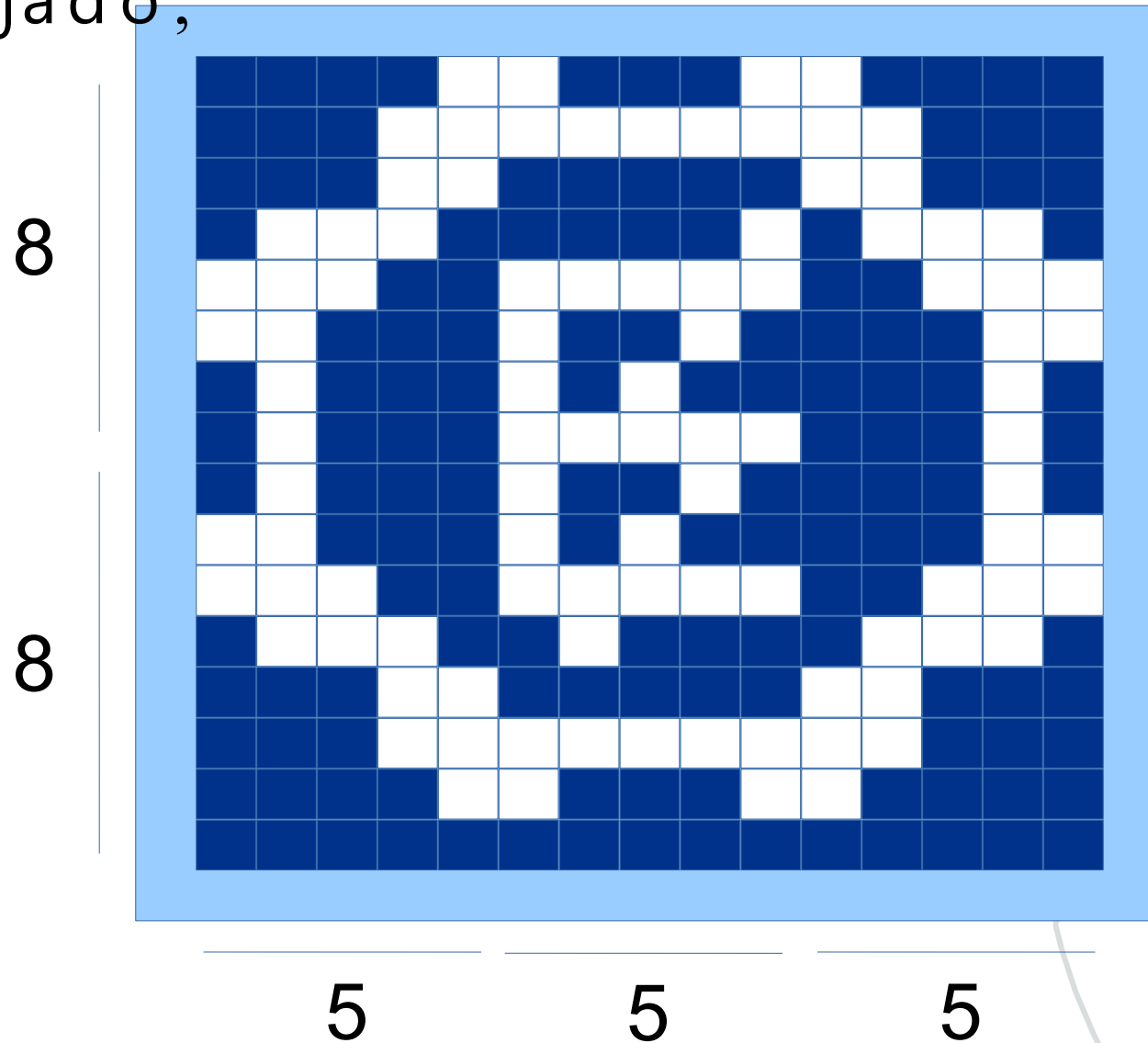


Passos para geração de uma imagem

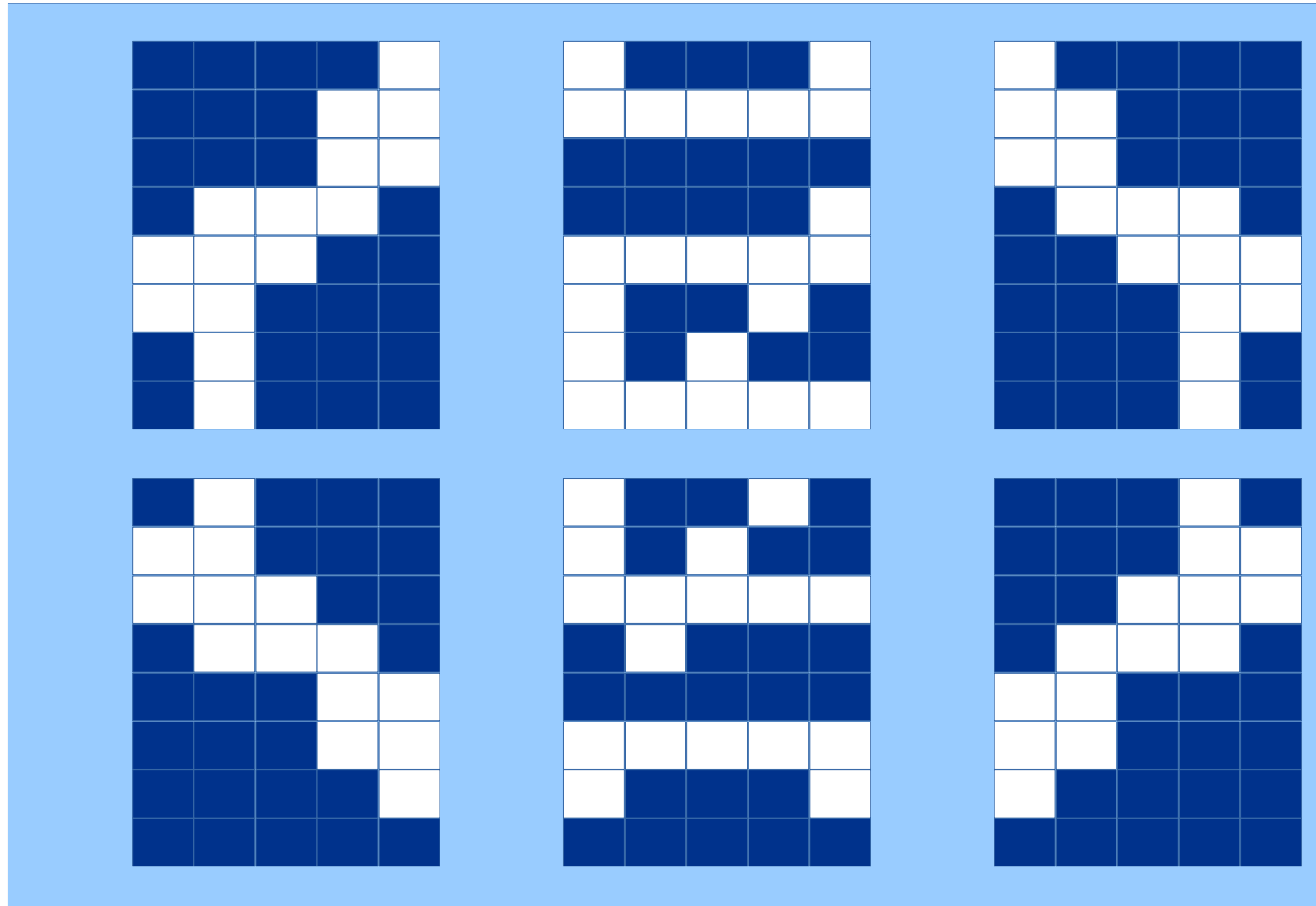
1. Criar uma imagem binária com o desenho desejado.
1. Segmentar a imagem em retângulos de 8x5.
1. Transcrever cada linha em binário/hexadecimal.
1. Gerar o código fonte.



- 1 - Criar uma imagem binária com o desenho desejado;



- 2 - Segmentar a imagem em retângulos de 8x5;



- 3 - Transcrever cada linha em binário/hexadecimal;

0x01	0	0	0	0	1
0x03	0	0	0	1	1
0x03	0	0	0	1	1
0x0E	0	1	1	1	0
0x1C	1	1	1	0	0
0x18	1	1	0	0	0
0x08	0	1	0	0	0
0x08	0	1	0	0	0

0x11	1	0	0	0	1
0x1F	1	1	1	1	1
0x00	0	0	0	0	0
0x01	0	0	0	0	1
0x1F	1	1	1	1	1
0x12	1	0	0	1	0
0x14	1	0	1	0	0
0x1F	1	1	1	1	1

0x10	1	0	0	0	0
0x18	1	1	0	0	0
0x18	1	1	0	0	0
0x0E	0	1	1	1	0
0x07	0	0	1	1	1
0x03	0	0	0	1	1
0x02	0	0	0	1	0
0x02	0	0	0	1	0

| | | | | | |

0x08	0	1	0	0	0
0x18	1	1	0	0	0
0x1C	1	1	1	0	0
0x0E	0	1	1	1	0
0x03	0	0	0	1	1
0x03	0	0	0	1	1
0x01	0	0	0	0	1
0x00	0	0	0	0	0

0x12	1	0	0	1	0
0x14	1	0	1	0	0
0x1F	1	1	1	1	1
0x08	0	1	0	0	0
0x00	0	0	0	0	0
0x1F	1	1	1	1	1
0x11	1	0	0	0	1
0x00	0	0	0	0	0

0x02	0	0	0	1	0
0x03	0	0	0	1	1
0x07	0	0	1	1	1
0x0E	0	1	1	1	0
0x18	1	1	0	0	0
0x18	1	1	0	0	0
0x10	1	0	0	0	0
0x00	0	0	0	0	0

- 4 - Gerar o código fonte.

//Cada linha é representada por um caracter

```
char logo[48] = {  
    0x01, 0x03, 0x03, 0x0E, 0x1C, 0x18, 0x08, 0x08, //0,0  
    0x11, 0x1F, 0x00, 0x01, 0x1F, 0x12, 0x14, 0x1F, //0,1  
    0x10, 0x18, 0x18, 0x0E, 0x07, 0x03, 0x02, 0x02, //0,2  
    0x08, 0x18, 0x1C, 0x0E, 0x03, 0x03, 0x01, 0x00, //1,0  
    0x12, 0x14, 0x1F, 0x08, 0x00, 0x1F, 0x11, 0x00, //1,1  
    0x02, 0x03, 0x07, 0x0E, 0x18, 0x18, 0x10, 0x00 //1,2  
};
```

lcdCommand(0x40); //Configura para a primeira posição de memória

//Envia cada uma das linhas em ordem

```
for(i=0; i<48; i++){  
    lcdChar(logo[i]);  
}
```

Resultado

...

```
lcdCommand(0x80); //Primeira linha
```

```
lcdChar(0);
```

```
LcdChar(1);
```

```
LcdChar(2);
```

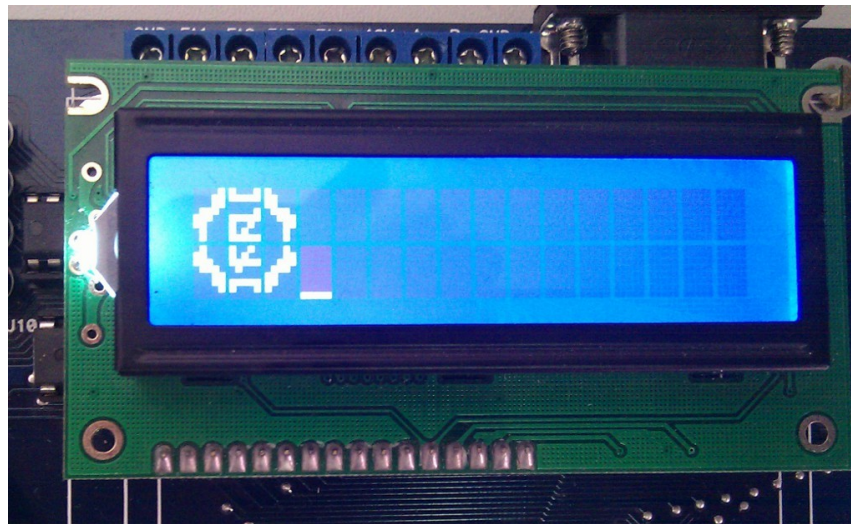
```
lcdCommand(0xC0); //Segunda linha
```

```
lcdChar(3);
```

```
LcdChar(4);
```

```
LcdChar(5);
```

...



Até a próxima

